INTEGRATED PROJECT REPORT

On

InterviewIQ

Submitted in partial fulfilment of the requirement for the Course

Full Stack Engineering (22CS037)

of

COMPUTER SCIENCE AND ENGINEERING

B.E. Batch-2023

in

MAY-2026

Submitted By: -

Maridul Walia	**2310991897**
Michael Dhiman	**2310991902**
Mohd Sufiyan	**2310991904**
Naman Mittal	**2310992588**

Under the Guidance of:

Dr. Astha Gupta

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY

PUNJAB

FULL STACK ENGINEERING
PROJECT REPORT

On

InterviewIQ

Submitted in partial fulfilment of the requirement for the Course

Full Stack Engineering (22CS037)

of

COMPUTER SCIENCE AND ENGINEERING

B.E. Batch-2023

in

MAY-2026

Submitted By: -

Maridul Walia	**2310991897**
Michael Dhiman	**2310991902**
Mohd Sufiyan	**2310991904**
Naman Mittal	**2310992588**

Under the Guidance of:

Dr. Astha Gupta

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY

PUNJAB

CERTIFICATE

This is to certify that the project entitled “InterviewIQ” has been submitted for the Bachelor of Computer Science Engineering at Chitkara University, Punjab during the academic semester January 2026-May 2026. It is a bonafide piece of project work carried out by Maridul Walia (2310991897), Michael Dhiman (2310991902), Mohd Sufiyan (2310991904), and Naman Mittal (2310992588) towards the partial fulfillment for the award of the course Full Stack Engineering (22CS037) under the guidance of Dr. Astha Gupta and supervision of the Department of Computer Science and Engineering.

Sign. of Project Guide:

Dr.Astha Gupta

Assistant Professor

Department of Computer Science and Engineering

CANDIDATE'S DECLARATION

We, Maridul Walia, Michael Dhiman, Mohd Sufiyan, and Naman Mittal, B.E.-2023 of Chitkara University, Punjab hereby declare that the Full Stack Engineering (22CS037) Project Report entitled "InterviewIQ" is an original work and the data provided in the study is authentic to the best of our knowledge. This report has not been submitted to any other institute for the award of any other course.

Sign. of Student 1	**Sign. of Student 2**	**Sign. of Student 3**	**Sign. of Student 4**
Name: Maridul Walia ID No.: 2310991897	Name: Michael Dhiman ID No.: 2310991902	Name: Mohd Sufiyan ID No.:2310991904	Name: Naman Mittal ID No.:2310992588

Place: Chitkara University, Punjab

Date: _____

ABSTRACT

InterviewIQ is an AI-powered interview preparation platform that creates personalized mock interviews from a user's uploaded resume. The project addresses the limitation of generic interview practice by using resume parsing, role selection, difficulty configuration, and AI-generated questions to create interview sessions that reflect the candidate's actual skills, projects, and preparation goals. The system supports authentication, resume management, configurable mock interviews, real-time answer evaluation, historical performance analytics, and recommendations for improvement.

The platform is implemented using a modern full-stack architecture. The frontend is built with React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, Framer Motion, and Recharts. The backend is built with Node.js and Express.js using a controller-service-route structure, while MongoDB and Mongoose are used for persistence. Google Gemini AI is used for both question generation and answer evaluation. The system stores resume text, generated interview sessions, questions, answers, scores, missing points, feedback, and analytics records.

The outcome of the project is a web application that enables candidates to upload resumes, configure interview sessions, attempt technical, HR, behavioral, and aptitude questions, and receive structured feedback. InterviewIQ therefore bridges the gap between static interview preparation and personalized, evidence-based interview practice.

Keywords: Interview Preparation, Artificial Intelligence, Resume Parsing, Mock Interview, Gemini AI, MERN Stack, MongoDB, React, Node.js, Express.js, Analytics

ACKNOWLEDGEMENT

It is our pleasure to express gratitude to various people who directly or indirectly contributed to the development of this project and influenced our thinking, behavior, and work during the course of study.

We express our sincere gratitude to the Department of Computer Science and Engineering, Chitkara University, Punjab for providing us the opportunity to undertake this Integrated Project as a part of the curriculum of Full Stack Engineering (22CS037).

We are thankful to our Project Guide, Dr. Astha Gupta, for support, cooperation, and motivation during the development of InterviewIQ. The guidance helped us understand the importance of system architecture, modular development, software testing, security, and user-centered design in a full-stack application.

Lastly, we would like to thank our parents, friends, and peers for their moral support and suggestions that helped improve the quality of this work.

Index

S.no	Section	Page No.
1	Cover Page	1
2	Title Page	2
3	Certificate	3
4	Declaration	4
5	Abstract	5
6	Acknowledgement	6
7	Contents / Index	7
8	List of Figures	8
9	List of Tables	9
10	Notations	10
11	Executive Summary	11
12	CHAPTER 1: Abstract and Keywords	12
13	CHAPTER 2: Introduction to the Project	13-14
14	CHAPTER 3: Software and Hardware Requirement Specification	15-17
15	CHAPTER 4: Database Analysis, Design and Implementation	18-20
16	CHAPTER 5: Program Structure Analysis and GUI Construction	21-26
17	CHAPTER 6: Code Implementation and Database Connections	27-28
18	CHAPTER 7: System Testing	29-31
19	CHAPTER 8: Limitations	32
20	CHAPTER 9: Conclusion	33
21	CHAPTER 10: Future Scope	33
22	CHAPTER 11: Bibliography/References	35
23	Appendix / Annexure	36-38

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	InterviewIQ High Level Architecture	16
Figure 4.1	Database Model Relationship Diagram	17
Figure 5.1	AI Question and Evaluation Workflow	21
Figure 5.7.1	Login	22
Figure 5.7.2	Register	22
Figure 5.7.3	Dashboard	23
Figure 5.7.4	Resume Upload Option	23
Figure 5.7.5	Resume Uploaded	23
Figure 5.7.6	Interview Configuration	23
Figure 5.7.7	AI-generated Interview Questions	24
Figure 5.7.8	AI-generated Interview Questions Mock Mode	24
Figure 5.7.9	Analytics	25

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Users of System	13
Table 3.1	Programming / Working Environment	14
Table 3.2	Hardware Requirements	15
Table 4.1	Database Models	17
Table 5.1	GUI Modules	21
Table 6.1	API Endpoints	27
Table 7.1	System Test Cases	29

NOTATIONS

Notation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DOCX	Microsoft Word document file format
JWT	JSON Web Token
MERN	MongoDB, Express.js, React, Node.js
REST	Representational State Transfer
SPA	Single Page Application
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
ODM	Object Data Modeling

Executive Summary

InterviewIQ is a full-stack AI interview preparation system that helps candidates practice interviews based on their own resumes. The platform allows a user to create an account, upload a PDF or DOCX resume, configure an interview by role, difficulty, category, and question count, and then attempt questions generated with Gemini AI. Every answer is evaluated by AI and stored with a score, detailed feedback, missing points, and improvement suggestions.

The system is divided into a React TypeScript frontend, an Express.js backend, MongoDB database, and Google Gemini AI service integration. The backend follows a modular architecture where routes, controllers, services, middleware, and models are separated to make the system maintainable. The frontend uses reusable UI components, React Router for navigation, and React Query with Axios for server-state handling.

The project also includes persistent InterviewSession records, which reduce repeated AI generation by storing generated questions for a user and session configuration. This supports reuse, analytics, and cost control. The answer evaluation records are then used to build analytics for weak area detection and performance tracking.

The report explains the project background, problem statement, requirement specification, methods, working environment, database design, program structure, GUI flow, implementation details, testing strategy, limitations, conclusion, and future scope.

Chapter 1
Abstract And Keywords

1.1 Project Abstract

InterviewIQ is designed as a personalized interview preparation platform for students, job seekers, and early-career developers. Traditional interview preparation applications usually provide static question banks that do not consider the candidate's exact resume, project history, technology stack, or target role. InterviewIQ solves this problem by analyzing the resume and generating interview questions that are directly related to the candidate's skills, experience, and desired difficulty level.

The application supports technical, HR, behavioral, and aptitude question categories. A candidate may select a mock interview mode with timer-based practice to simulate actual interview pressure, or use normal practice mode for self-paced learning. The answer evaluation module gives a score out of 10, constructive feedback, missing points, and improvement suggestions.

The project demonstrates important concepts of full-stack engineering, including authentication, protected API routes, file upload handling, PDF and DOCX parsing, structured AI prompts, JSON response validation, MongoDB schema design, analytics generation, and component-based frontend development.

1.2 Keywords

- Artificial Intelligence Interview Preparation
- Resume Driven Question Generation
- Mock Interview Platform
- React TypeScript Frontend
- Node.js Express Backend
- MongoDB and Mongoose
- Google Gemini AI
- Answer Evaluation and Analytics

1.3 Project Deliverables

- A working React-based user interface for authentication, resume upload, dashboard, interview execution, and analytics.
- A RESTful backend API with JWT authentication, upload middleware, validators, business services, and database models.
- AI-based question generation using resume context, role, difficulty, and configured category distribution.
- AI-based answer evaluation with score, feedback, missing points, and improvement suggestions.
- MongoDB persistence for users, resumes, questions, answers, and interview sessions.
- Automated test simulation for login, protected routes, answer submission, analytics, and route switching.

Chapter 2
Introduction To The Project

2.1 Background

Interviews are a major selection stage for internships and jobs. Candidates often struggle not because they lack knowledge, but because their preparation is not aligned with their own resume or the role they are applying for. For example, a student with React and Node.js projects may require frontend, backend, database, and system design questions, while another student may need questions focused on SQL, operating systems, computer networks, or aptitude.

Existing question-bank applications provide common questions, but they cannot deeply analyze a candidate's resume and adapt the session to the candidate's profile. They also rarely provide immediate structured feedback with missing concepts. This creates a gap between preparation and real-world interview expectations.

InterviewIQ was built to address this gap. It provides a resume-aware AI interviewer that can generate relevant questions and evaluate answers immediately. The system is useful for students who want repeated practice, measurable feedback, and a clear understanding of weak areas before actual interviews.

2.2 Problem Statement

The problem is to design and implement a full-stack web application that helps users prepare for interviews through personalized AI-based practice. Many existing interview preparation methods provide generic questions that do not match the user's resume, skills, projects, or target role. InterviewIQ solves this by allowing users to upload their resume, generate customized interview questions, attempt mock interviews, and receive real-time AI evaluation with scores, feedback, missing points, and improvement suggestions. The system also stores performance history and provides analytics to help users track progress over time. It must be secure, scalable, modular, and easy to run locally for development and testing.

2.3 Objectives

- To build a secure authentication system using signup, login, password hashing, and JWT-based protected routes.
- To enable resume upload in PDF and DOCX formats and extract text for AI context.
- To generate interview questions based on resume content, target role, selected difficulty, and question configuration.
- To support both timer-based mock interview mode and normal practice mode.
- To evaluate answers using AI and store feedback, missing points, and score.
- To display analytics that help users identify weak areas and track performance.
- To maintain a clean architecture with controller-service-route separation and reusable frontend components.

2.4 Scope of the Project

The scope of InterviewIQ includes user registration, login, resume management, AI-based question generation, mock interview execution, answer evaluation, result storage, analytics, and basic automated testing. The system is currently focused on web-based interview preparation and local development. Deployment, voice interviews, video analysis, and real-time conversational voice agents are considered future extensions.

2.5 Users of the System

User Type	Description	Major Activities
Candidate/Student	Primary user preparing for interviews	Upload resume, configure interview, answer questions, review analytics
Developer/Maintainer	Person maintaining the project	Run locally, test APIs, improve models and UI

Table 2.1 Users of System

2.6 Significance

The significance of InterviewIQ lies in its ability to provide contextual interview preparation instead of generic practice. Since the resume becomes the foundation of the generated session, candidates can practice questions that are more likely to be asked about their own projects and skills. The feedback mechanism makes the learning loop immediate and measurable.

Chapter 3
Software And Hardware Requirement Specification

3.1 Methods

The project was developed using an iterative full-stack development method. The system was divided into independent modules such as authentication, resume upload, AI question generation, answer evaluation, analytics, and UI routing. Each module was implemented and tested separately before integration.

- Requirement analysis was performed from the perspective of interview candidates and developers.
- Database schemas were designed before implementing controllers to ensure consistent storage.
- API routes were protected using JWT middleware and validated using backend validation libraries.
- The AI service was isolated in a service layer so that the AI provider can be replaced in the future.
- Frontend components were separated into pages, reusable UI components, hooks, and API services.
- Testing was performed through an automated script and manual end-to-end flows.

3.2 Programming/Working Environment

Layer	Technology / Tool	Purpose
Frontend	React 18, TypeScript, Vite	Build fast interactive SPA
Routing	React Router v6	Client-side page navigation
Forms	React Hook Form, Zod	Validated user input
Server State	TanStack React Query, Axios	API calls and caching
UI	Tailwind CSS, shadcn/ui, Radix UI	Responsive accessible interface
Animation	Framer Motion	Smooth UI transitions
Charts	Recharts	Analytics visualization
Backend	Node.js, Express.js	REST API server
Validation	Joi, express-validator	Request validation
File Upload	multer, pdf-parse, mammoth	Resume upload and parsing
Database	MongoDB, Mongoose	Data persistence and schema models
AI	Google Generative AI Gemini 1.5 Flash	Question generation and answer evaluation
Authentication	bcryptjs, jsonwebtoken	Password hashing and JWT sessions

Table 3.2 Working environment

3.3 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Dual-core processor	Quad-core or higher
RAM	4 GB	8 GB or higher
Storage	1 GB free storage	5 GB free storage
Network	Internet for Gemini API	Stable broadband connection
Display	1366x768	Full HD display

Table 3.3 Hardware Requirements

3.4 Requirements to Run the Application

- Node.js v18 or higher
- MongoDB local instance or MongoDB Atlas URI
- Google Gemini API Key
- Modern browser such as Chrome, Edge, or Firefox

3.5 Local Run Steps

The application can be installed and started from the root directory. The root package is configured to install and run the frontend and backend together.

git clone https://github.com/maridulwalia/InterviewIQ

The application can be installed and started from the root directory. Developers should create a backend .env file with PORT, MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN, and GEMINI_API_KEY before starting the servers.

cd backend

cp .env.example .env

npm run dev

In another terminal for frontend

cd frontend

npm run dev

Or from project root

npm run dev

3.6 Environment Variables

PORT=5000

MONGO_URI=mongodb://localhost:27017/interviewiq

JWT_SECRET=your_super_secret_jwt_key_change_this_in_production

JWT_EXPIRES_IN=7d

GEMINI_API_KEY=your_gemini_api_key_here

3.7 System Architecture Diagram

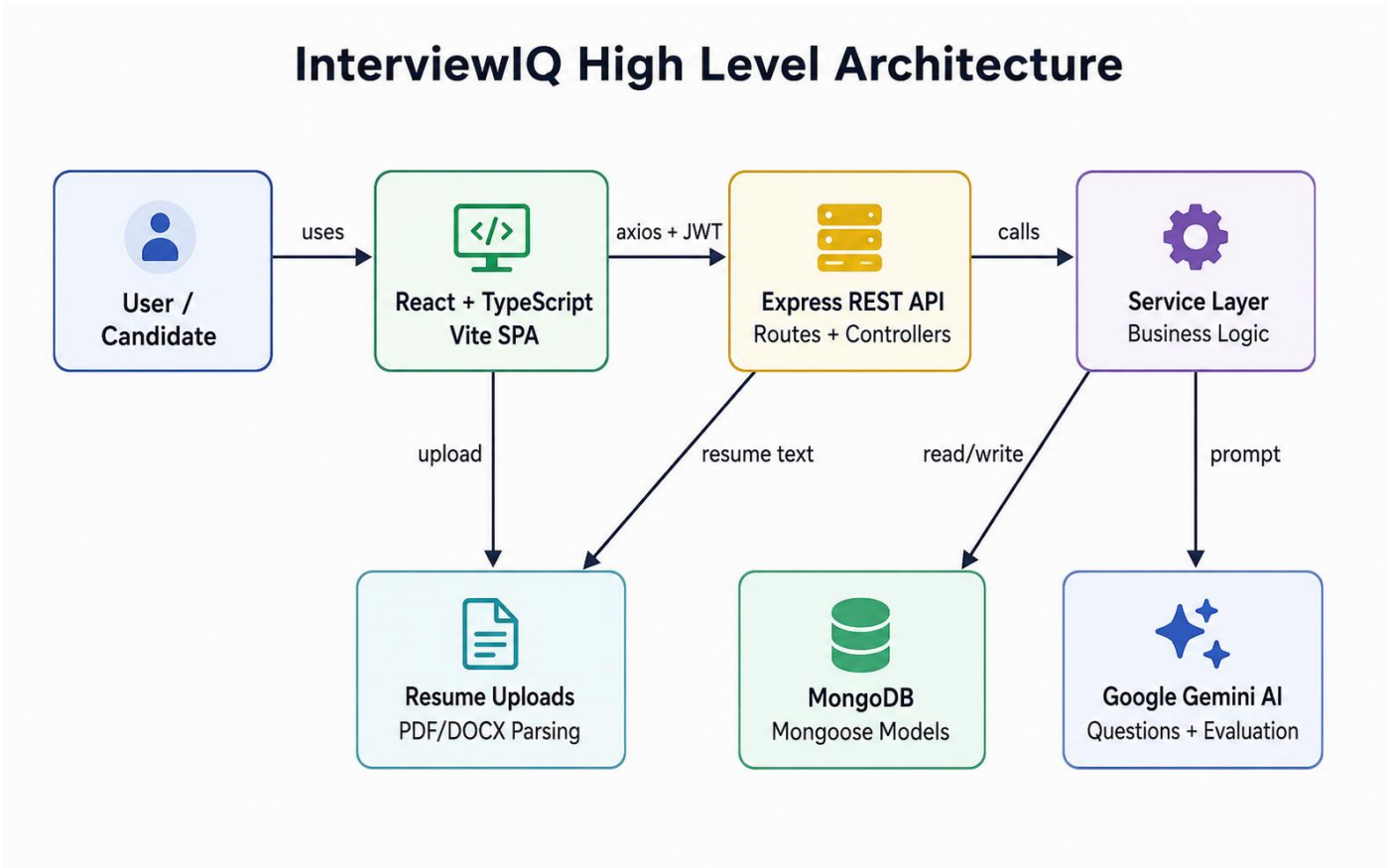

Figure 3.1: InterviewIQ High Level Architecture

The architecture follows a decoupled client-server model. The React frontend communicates with the Express backend through Axios requests. The backend validates requests, applies authentication middleware, invokes services, stores data in MongoDB, and communicates with Gemini AI when questions or answer evaluations are required.

Chapter 4
Database Analysis, Design And Implementation

4.1 Database Selection

MongoDB was selected because InterviewIQ stores semi-structured data such as resume metadata, extracted text, AI-generated questions, answer feedback, missing points, and session configuration. These records can vary in size and structure, making a document database suitable for rapid development and flexible data modeling.

4.2 Database Models

Model	Main Fields	Purpose
User	name, email, passwordHash	Stores identity and authentication information. Passwords are hashed before storage
Resume	userId, originalName, fileUrl, extractedText	Stores uploaded resume, details and extracted text used for AI context
Question	text, type, category	Stores predefined fallback questions
Answer	userId, questionId, sessionId, answerText, score, feedback, missingPoints, improvementSuggestions	Stores user attempts and structured AI evaluation output
InterviewSession	userId, role, difficulty, questionConfig, questions, generatedByAI	Stores generated interview sessions and question configuration for reuse

4.3 Entity Relationship Diagram

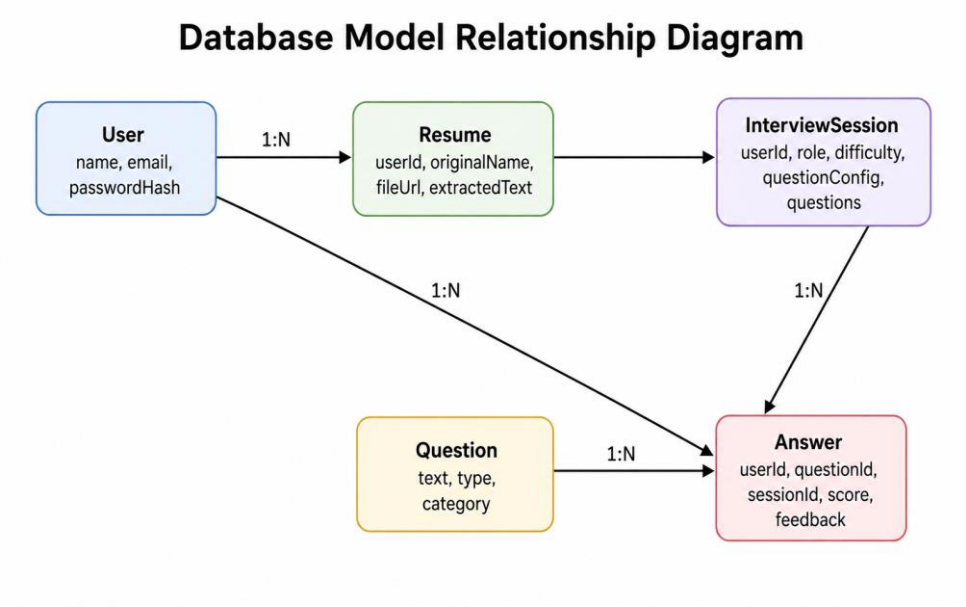

Figure 4.1: Database Model Relationship Diagram

4.4 User Model

The User model represents each registered candidate. It stores name, email, and hashed password. The email field is expected to be unique so that the same person cannot register multiple accounts with the same email address. Password hashing using bcryptjs protects credentials even if database records are exposed.

4.5 Resume Model

The Resume model is central to personalization. When a user uploads a PDF or DOCX file, the backend stores metadata such as original filename and file URL or path. The extractedText field stores parsed resume content. This text is later supplied to the AI prompt to generate role-specific and resume-aware interview questions.

4.6 Question Model

The Question model is a static repository of predefined fallback questions. It contains fields such as text, type, and category. This helps the platform continue operating even when AI generation fails due to timeout, quota, malformed output, or invalid credentials.

4.7 Answer Model

The Answer model stores each user attempt. It links the user, question, and interview session. It also stores answerText, score, feedback, missingPoints, and improvementSuggestion. These records become the source of analytics and weak area identification.

4.8 InterviewSession Model

The InterviewSession model represents a complete interview event. It stores the selected role, difficulty, question distribution, generated questions, and generatedByAI flag. It also supports session caching, meaning already generated questions can be reused instead of repeatedly calling the AI service for the same configuration.

4.9 Indexing and Performance

A compound index on userId, role, difficulty, and configuration improves the speed of locating previously generated sessions. This design reduces duplicate AI calls, lowers storage growth, and keeps interview history tied to the user account. Indexes on userId fields also improve query performance for resume, answer, and analytics retrieval.

4.10 Data Privacy Considerations

Resume text may contain sensitive personal information such as phone numbers, education history, projects, and skills. Therefore, protected routes and user-specific queries are important. Each resume, session, and answer should be tied to the authenticated userId to prevent unauthorized access to another user's data.

Chapter 5
Program Structure Analysis And GUI Construction

5.1 Root Folder Structure

InterviewIQ/

```
├── backend/          # Node.js Express API
├── frontend/         # React Vite Application
├── package.json      # Root package with concurrent scripts
└── README.md
```

5.2 Backend Folder Structure

backend/

```
├── config/           # Database config and seed scripts
├── controllers/      # HTTP handlers: auth, resume, question, answer, analytics
├── middleware/       # JWT auth, Multer upload, Joi validators
├── models/           # Mongoose schemas
├── routes/           # Express routers
├── services/         # Business logic: aiService, analyticsService, resumeService
├── uploads/          # Temporary local file storage
└── server.js         # Application entry point
```

5.3 Frontend Folder Structure

frontend/

```
├── public/
└── src/
    ├── components/   # Reusable UI components
    ├── hooks/        # Custom hooks
    ├── lib/          # Utility functions
    ├── pages/        # Dashboard, Interview, Login, Resume Management
    ├── services/     # Axios API wrappers
    ├── App.tsx
    └── main.tsx
```

5.4 GUI Modules

GUI Module	Purpose	Main User Action
Signup/Login	Authenticate user and store JWT	Create account or login
Dashboard	Central control page	Select role, difficulty, and question distribution
Resume Management	Upload and manage resumes	Upload PDF/DOCX resume and view parsed status
Interview Page	Run mock or practice interview	Read questions and submit answers
Evaluation View	Show AI evaluation	Review score, feedback, missing points, suggestions
Analytics Dashboard	Display performance trends	Identify weak categories and recommendations

Table 5.1 GUI Modules

5.5 AI Workflow Diagram

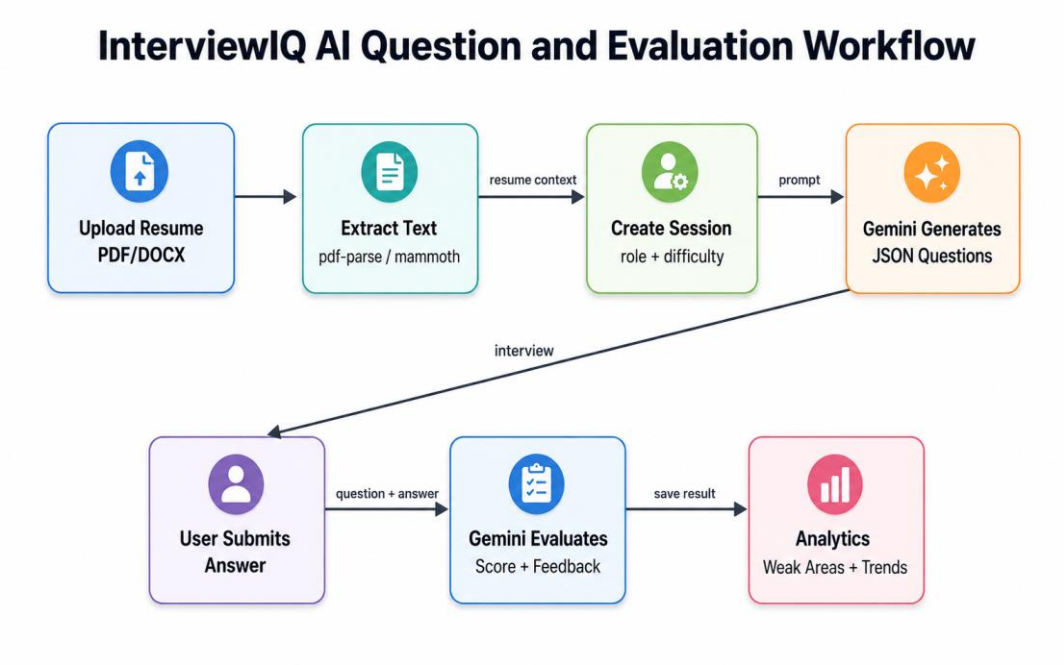

Figure 5.1: AI Question and Evaluation Workflow

5.6 Application Flow

1. User signs up or logs in and receives a JWT.
2. User uploads a current resume in PDF or DOCX format.
3. Backend parses the file and stores extracted text in the Resume model.
4. User configures a mock interview by selecting role, difficulty, and question distribution.
5. Backend checks whether a matching session already exists for that user and configuration.
6. If not cached, the AI service generates structured questions using Gemini AI.
7. User attempts questions in mock mode or practice mode.

8. Each answer is evaluated and stored.
9. Analytics are generated from historical scores and category performance.

5.7 Project Snapshots

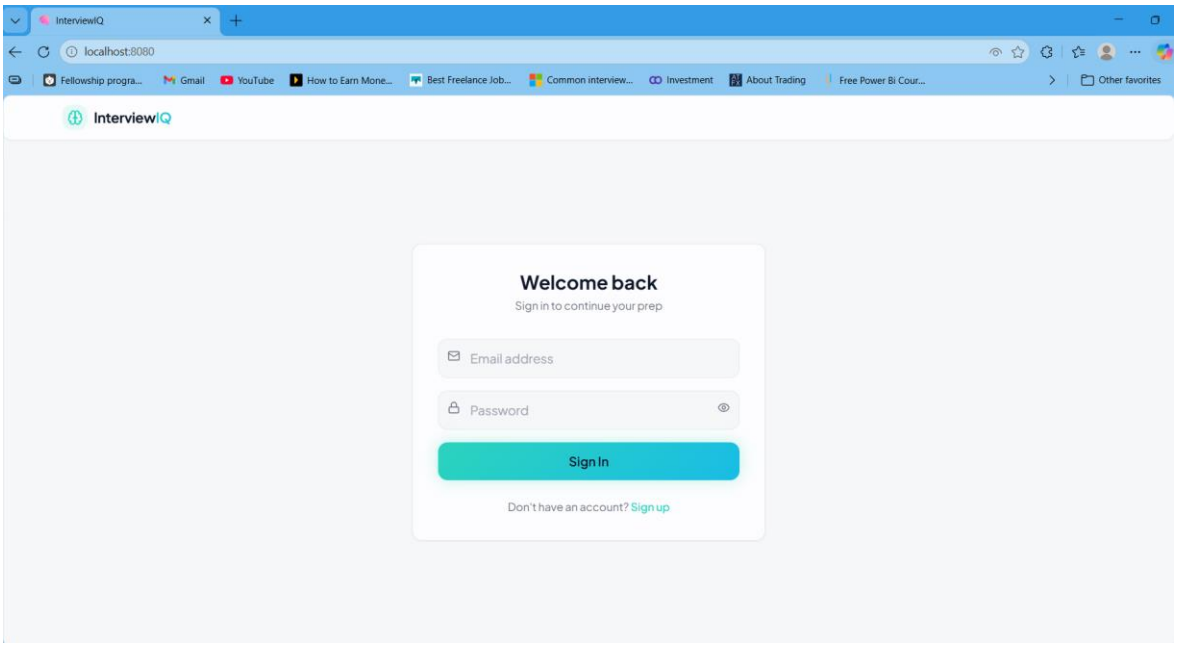

Figure 5.7.1 Login

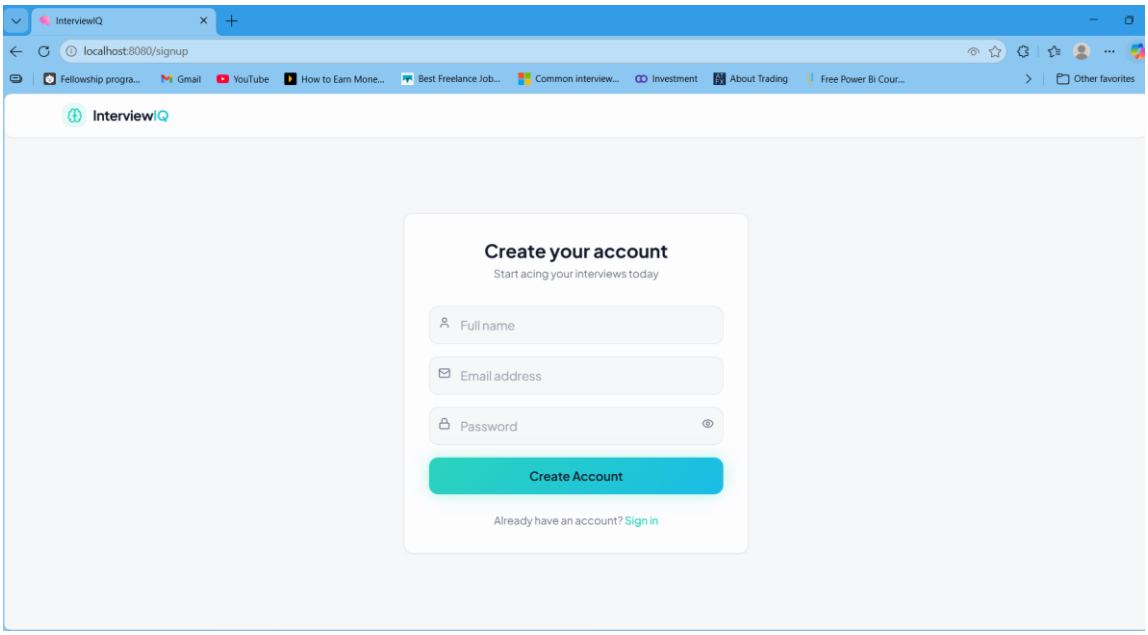

Figure 5.7.2 Register

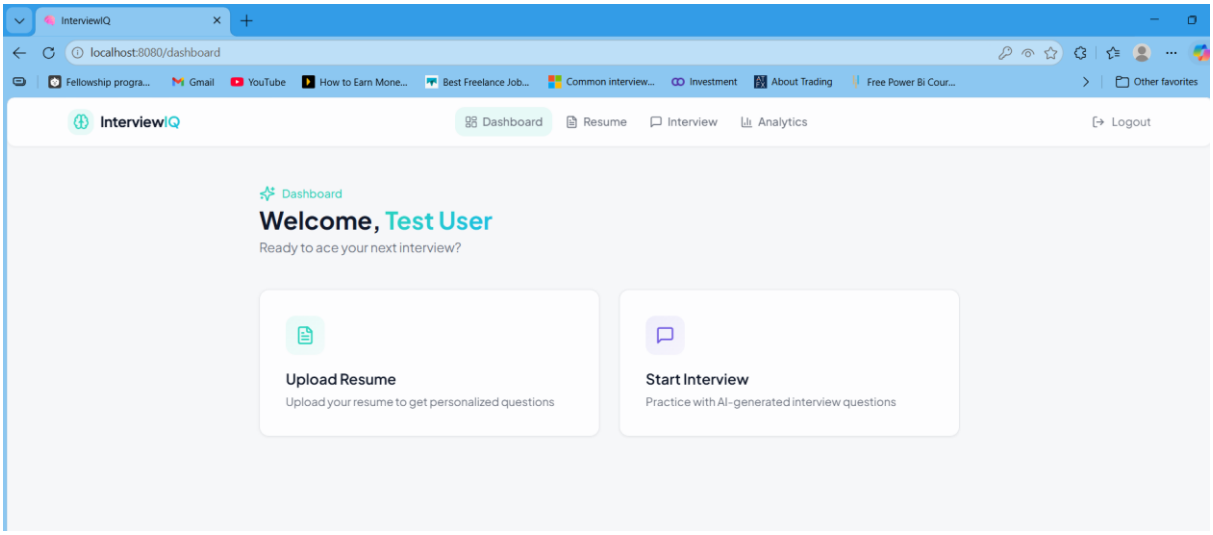

Figure 5.7.3 Dashboard

Figure 5.7.4 Resume Upload Option

Figure 5.7.5 Resume Uploaded

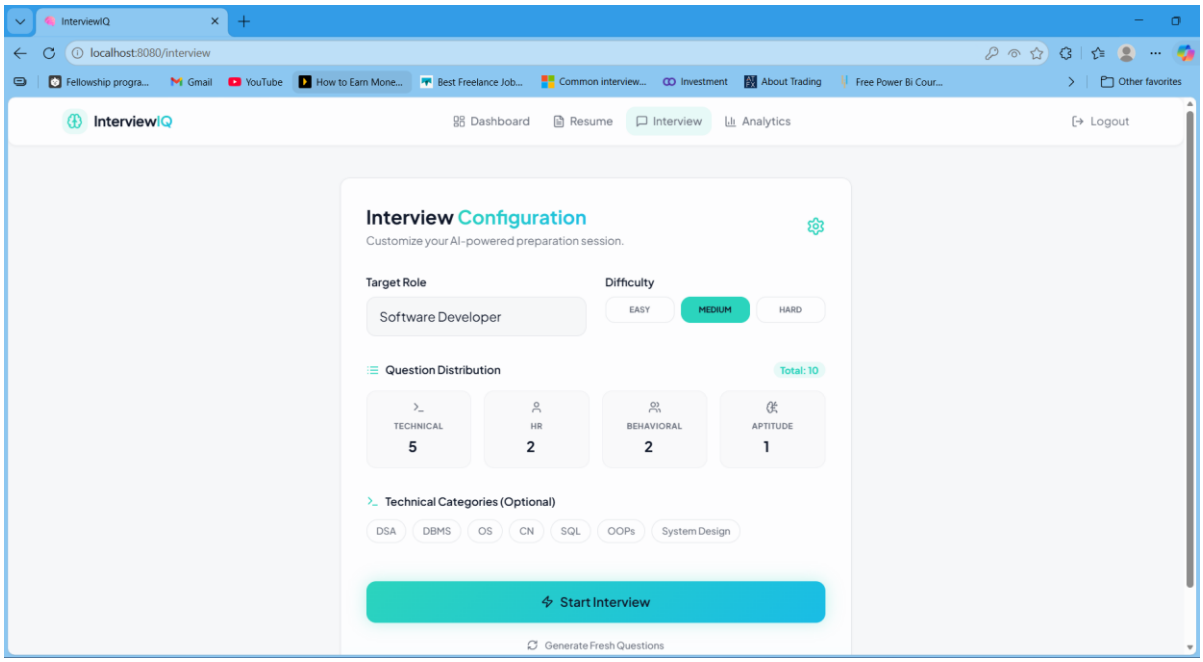

Figure 5.7.6 Interview Configuration

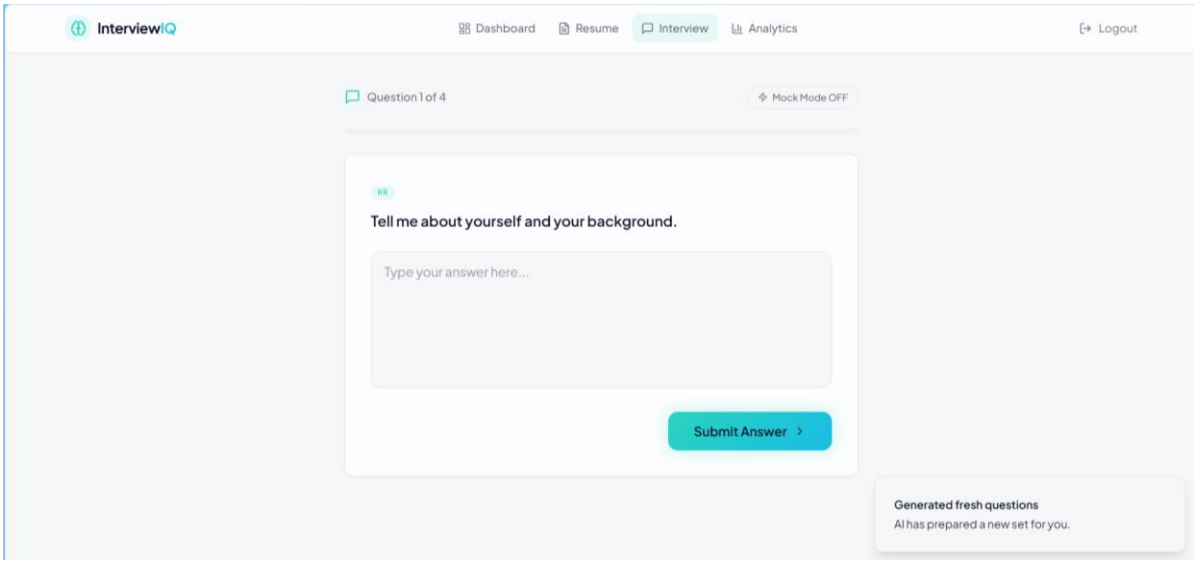

Figure 5.7.7 AI-generated Interview Questions

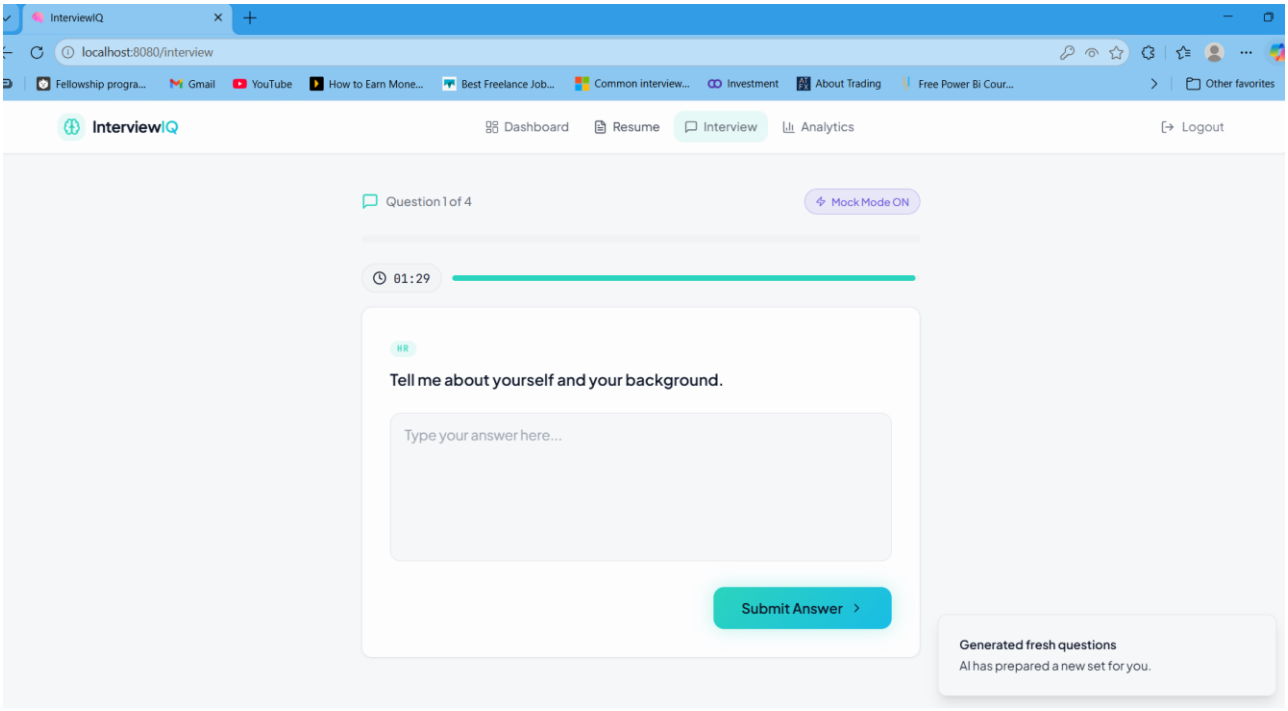

Figure 5.7.8 AI-generated Interview Questions Mock Mode

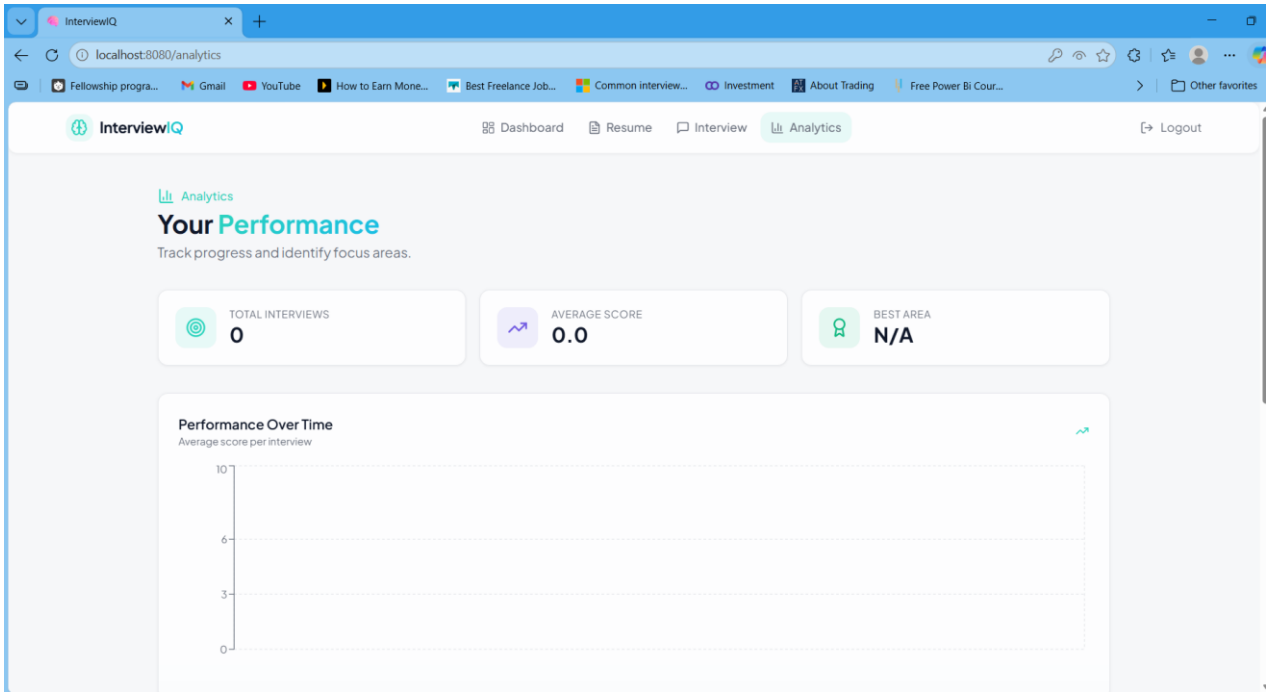

Figure 5.7.9 Analytics

5.8 UI Design Principles

- Consistency: reusable shadcn/ui components and Tailwind utility classes keep spacing, colors, and controls consistent.
- Accessibility: Radix UI primitives support keyboard navigation and accessible component behavior.
- Responsiveness: layout is designed for common desktop and laptop screens used by students.
- Feedback: answer evaluation provides immediate visual and textual feedback.
- Clarity: dashboard controls separate role, difficulty, question count, and category selection.

Chapter 6
Code Implementation And Database Connections

6.1 Backend Implementation Approach

The backend is implemented as a RESTful Express API. The entry point initializes the Express application, connects middleware, configures CORS, mounts route files, and starts the server on the configured port. Business logic is kept inside service files to avoid putting complex logic directly inside routes.

6.2 Controller-Service-Route Pattern

The route layer defines endpoint paths and middleware. The controller layer reads request data and returns HTTP responses. The service layer performs core logic such as parsing resumes, building AI prompts, reading and writing database records, and calculating analytics. This separation makes the codebase easier to test and maintain.

6.36.3 Authentication Implementation

1. Signup receives name, email, and password.
2. Password is hashed using bcryptjs before being stored.
3. Login verifies email and password.
4. A JWT is generated and returned to the frontend.
5. Protected routes require Authorization: Bearer <token>.
6. The auth middleware decodes the token and attaches the user identity to the request.

6.4 Resume Upload and Parsing

Resume upload uses multer to accept files from multipart/form-data. PDF files are parsed using pdf-parse and DOCX files are parsed using mammoth. Extracted text is stored in the Resume collection and is used as the primary context for generating personalized questions.

6.5 AI Question Generation

The AI service builds a prompt containing the target role, selected difficulty, question configuration, and resume text. The prompt instructs Gemini AI to return strict JSON. The service includes a JSON extraction step that removes markdown code fences and attempts to parse the returned response safely.

6.6 AI Answer Evaluation

When a user submits an answer, the backend sends the original question, the answer text, and resume context to Gemini AI. The response is expected to include a score out of 10, feedback, missing points, and improvement suggestions. The result is saved in the Answer model and later used by the analytics module.

6.7 Database Connection

The backend connects to MongoDB using Mongoose and the MONGO_URI environment variable. A local MongoDB instance can be used during development, while MongoDB Atlas can be used in production. Mongoose schemas provide model-level structure and validation for all stored records.

```
// Conceptual database connection

mongoose.connect(process.env.MONGO_URI)

  .then(() => console.log('MongoDB connected'))

  .catch((err) => console.error('MongoDB connection error', err));
```

6.8 API Endpoints

Method	Endpoint	Auth	Purpose
POST	/api/auth/signup	No	Register new user
POST	/api/auth/login	No	Authenticate user
POST	/api/resume/upload	Yes	Upload and parse resume
GET	/api/resume/me	Yes	Get active user resume
GET	/api/resume	Yes	Get all user resumes
DELETE	/api/resume/:id	Yes	Delete resume
POST	/api/questions	Yes	Generate custom questions
GET	/api/questions/all	Yes	Get predefined questions
POST	/api/questions/mock	Yes	Generate mock interview session
POST	/api/answers	Yes	Submit answer and receive AI evaluation
GET	/api/answers	Yes	Get historical answers
GET	/api/analytics	Yes	Get user performance analytics

Table 6.1 API Endpoints

6.9 Session Caching Logic

The InterviewSession model allows generated questions to be tied to the user, role, difficulty, and question configuration. Before calling Gemini AI, the backend can search for an existing session with the same configuration. If a matching session exists, the stored questions can be reused to reduce duplicate AI calls, control storage growth, and preserve interview history.

6.10 Error Handling

- Invalid login returns an authentication error without exposing whether password hashing failed.
- Invalid or missing JWT returns unauthorized access.
- Unsupported resume file types are rejected by upload validation.

- AI response parsing failure can trigger fallback questions.
- Database errors are handled by controllers and logged for debugging.
- Frontend displays user-friendly messages for network, validation, or server failures.

Chapter 7
System Testing

7.1 Testing Objective

Testing verifies that the system works correctly across authentication, token persistence, protected API access, resume upload, question generation, answer submission, analytics retrieval, and route switching. Since InterviewIQ depends on both database state and AI service responses, testing covers both normal flows and expected failure conditions.

7.2 Automated Test Simulation

The project includes script-tests.js to simulate the core application flow. This script checks login, token persistence, protected route access, answer submission, analytics retrieval, and route-switch stress behavior.

node script-tests.js

7.3 Test Cases

Test Case	Input / Action	Expected Result	Status
Login Test	Valid email and password	JWT token generated	Pass
Invalid Login	Wrong password	401 or validation error	Expected failure
Token Persistence	Switch routes after login	Token remains available	Pass
Resume Upload	Upload PDF/DOCX	Resume stored and text extracted	Pass when file is valid
Questions API	Call protected endpoint with token	Questions returned or generated	Pass
Answer Submission	Submit answerText and questionId	Score and feedback stored	Pass
Analytics	Request analytics endpoint	Historical metrics returned	Pass
Route Stress	Rapidly query endpoints	Token not invalidated	Pass

7.4 Functional Testing

- Authentication flow was tested through signup, login, and protected route access.
- Resume upload was tested with supported PDF and DOCX formats.
- Question generation was tested with role, difficulty, and question configuration.
- Mock mode was tested with timer-based session behavior.
- Practice mode was tested without time pressure.
- Answer evaluation was tested for score, feedback, missing points, and suggestions.
- Analytics were tested by retrieving saved answer records.

7.5 Security Testing

- Protected APIs should reject requests without a valid JWT.
- Passwords are not stored in plain text.
- Resume and answer records must be scoped by authenticated userId.
- Environment secrets such as JWT_SECRET and GEMINI_API_KEY are stored in backend .env and should not be committed.
- Upload middleware should restrict file type and size to reduce abuse.

7.6 Performance Testing Considerations

Performance testing should focus on repeated AI generation, database query speed, resume parsing time, and analytics response time. The most expensive operation is AI generation, so session caching and reuse are important architectural decisions. Database indexes on userId and session configuration fields reduce lookup time as the number of users and sessions increases.

7.7 Testing Observations

The system is suitable for development testing using local MongoDB, backend on port 5000, and Vite frontend on port 5173. For production readiness, additional tests should be added for rate limiting, file size boundaries, AI quota failures, and concurrent session generation requests.

Chapter 8
Limitations

- AI output quality depends on the quality of the uploaded resume, selected role, question configuration, and clarity of the generated prompt.
- Gemini API quota or network failure may temporarily affect question generation and evaluation.
- PDF parsing may fail for scanned PDFs or resumes with complex formatting because OCR is not included.
- The current version focuses on text-based interviews and does not yet support voice or video interviews.
- Analytics are based on stored answer records and become more useful only after multiple sessions.
- Local file upload storage must be replaced or hardened with cloud storage, access control, and cleanup policies for production deployment.
- The system requires careful prompt and JSON parsing safeguards because AI responses can sometimes deviate from strict structure.

8.1 Risk Mitigation

Fallback questions, session caching, validation, protected routes, and structured JSON parsing reduce major risks. Future production deployment should add rate limiting, secure cloud storage, monitoring, logging, and stronger automated test coverage.

Chapter 9
Conclusion

InterviewIQ successfully demonstrates a full-stack AI-powered interview preparation platform. It combines resume parsing, configurable mock interviews, AI-generated questions, answer evaluation, and analytics into one user-centered web application. The project applies core full-stack engineering principles such as modular backend architecture, React component-based frontend development, RESTful APIs, MongoDB schema design, authentication, validation, and third-party AI integration.

The system solves the main problem of generic interview preparation by generating questions that are influenced by the candidate's resume and selected role. It also gives immediate structured feedback, which helps users improve their future responses. The analytics module makes progress measurable by recording historical performance and identifying weak areas.

Overall, InterviewIQ is a practical and extensible platform. Its service-based backend makes it possible to replace or add AI providers, while the frontend architecture supports new pages and features. With additional production hardening and future enhancements such as voice and video interviews, the project can evolve into a stronger career preparation tool.

Chapter 10
Future Scope

- Speech-to-text integration so candidates can answer using a microphone.
- Video interview mode using WebRTC to record responses and optionally evaluate body language or eye contact.
- Adaptive difficulty that changes question difficulty based on rolling average performance.
- Real-time conversational voice agent that reads questions aloud and conducts a more natural mock interview.
- Dockerization using Docker and docker-compose for one-click local or server deployment.
- Cloud storage for resumes instead of temporary local uploads.
- Admin dashboard for managing predefined question banks, reviewing usage metrics, and monitoring system health if administrative controls are added in the future.
- Expanded technical categories such as DSA, SQL, computer networks, operating systems, DBMS, aptitude, React, Node.js, and system design.
- Detailed report export in PDF format for every completed interview session.
- Role-specific interview templates for frontend, backend, full-stack, data analyst, DevOps, and software engineer roles.

Chapter 11
Bibliography / References

1) Project README: InterviewIQ - AI Interview Preparation Platform.
2) React Documentation - Component-based user interface development.
3) Vite Documentation - Frontend tooling and development server.
4) Express.js Documentation - REST API and middleware development.
5) MongoDB and Mongoose Documentation - Document database modeling and schema design.
6) Google Generative AI Documentation - Gemini API integration for generation and evaluation.
7) JWT and bcryptjs Documentation - Authentication, authorization, and password hashing.
8) Tailwind CSS, shadcn/ui, Radix UI, Framer Motion, and Recharts Documentation - UI design, animation, accessibility, and analytics visualization.

Appendix A
Sample Api Requests

A.1 Login Request

POST /api/auth/login

Content-Type: application/json

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

A.2 Resume Upload Request

POST /api/resume/upload

Authorization: Bearer <token>

Content-Type: multipart/form-data

file: resume.pdf

A.3 Generate Mock Session Request

POST /api/questions/mock

Authorization: Bearer <token>

```
{
  "role": "Full Stack Developer",
  "difficulty": "MEDIUM",
  "config": {
    "technical": 5,
    "hr": 2,
    "behavioral": 2,
    "aptitude": 1
```

```
  }
}
```

A.4 Submit Answer Request

POST /api/answers

Authorization: Bearer <token>

```
{
  "questionId": "<question-id>",
  "sessionId": "<session-id>",
  "answerText": "My answer goes here..."
}
```

Appendix B
Glossary

Term	Description
Resume-Aware Question	Question generated using the user's resume as context.
Mock Mode	Interview mode with a timer to simulate real interview pressure.
Practice Mode	Interview mode without timer for self-paced practice.
Session Caching	Reusing generated questions for the same user and configuration.
Missing Points	Key concepts absent from the user's submitted answer.
Improvement Suggestions	Actionable AI feedback to improve future answers.